

Go API rutracker Search — Root-Cause Analysis (2026-07-02)

Isolated (no-emulator) root-cause of the autonomous-QA keystone failure: goapi backend × rutracker search shows **"problem reaching the trackers"** / **"Search failed"** after on-device onboarding login was fixed.

- **Environment:** Linux x86_64, podman 5.7.1, Mullvad VPN egress. Go API brought up via `BUILD_AH_FORMAT=docker ./start.sh` (profile `api-go`), version **2.3.33** / **build 2333**, listening :8443 (HTTP/3 + HTTP/2), `/health` → `{"status":"alive"}`.
- **Method:** black-box curl against the running container + source read of the search route, auth middleware, and rutracker client.
- **\$11.4.6 vocabulary:** claims below are backed by captured HTTP responses. Items I could not observe in isolation are marked UNCONFIRMED:.
- **\$6.H:** no credential/key/cookie VALUES are recorded here. Scratch files that held a real `bb_session` were deleted after the run (verified 0 residue).

1. The search route + auth model (source facts)

The Go API exposes rutracker search on **two** routes, both built by `internal/router/router.go Build()`:

Route	Handler	Registered
GET <code>/v1/:provider/search</code> (modern, provider-agnostic — the keystone path)	<code>internal/handlers/v1/search.go GetSearch</code>	<code>v1handlers.Register</code> , per-route <code>ProviderMiddleware(reg, CapSearch)</code>
GET <code>/search</code> (legacy rutracker-specific)	<code>internal/handlers/</code>	<code>handlers.Register</code>

Route	Handler	Registered
	search.go GetSearch	

Both routes sit behind TWO independent auth gates:

Gate 1 — Lava-Auth per-instance client key (candidate b)

internal/auth/middleware.go NewMiddleware, mounted globally at router.go:106 BEFORE the v1 group. It reads the header named by LAVA_AUTH_FIELD_NAME (Lava-Auth), base64-decodes it to a 16-byte UUID blob, HMAC-SHA256's it with LAVA_AUTH_HMAC_SECRET, and looks the hash up in LAVA_AUTH_ACTIVE_CLIENTS (.env, CSV of name:uuid).

- missing header → **401 {"error":"unauthorized"}** (middleware.go:57-60)
- malformed / unknown → 401 (+ backoff-ladder increment)
- retired hash → 426.

/health, /ready, /providers are mounted BEFORE this middleware and are public.

Gate 2 — rutracker session cookie forwarded via Auth-Token (upstream auth)

The Go API **does NOT log into rutracker on its own for search**. It is a pure **cookie pass-through**:

- v1 path: parseCredentials(c) → auth.ProviderCredentials → ParseAuthToken (internal/auth/multiprovider.go) parses Auth-Token: rutracker:cookie:bb_session=... (or a legacy bare cookie) into provider.Credentials{Type:"cookie", CookieValue:...}. The rutracker ProviderAdapter.Search (internal/rutracker/provider.go:67) calls credToCookie(cred) → the cookie string (empty unless Type=="cookie").

- Client.GetSearchPage (internal/rutracker/search.go:229-236)
- short-circuits:**

```
if cookie == "" {
    return nil,
        ErrUnauthorized // → provider.ErrUnauthorized → HTTP
        401 {}
}
```

- With a cookie, it does GET /forum/tracker.php?nm=<query>&... with Cookie: <value> and parses the HTML (windows-1251 → UTF-8).

Rutracker credentials the Go API itself uses: *none*. internal/config/config.go reads only LAVA_API_RUTRACKER_URL (default https://rutracker.org/forum). There is no RUTRACKER_USERNAME/RUTRACKER_PASSWORD read anywhere in the Go config — those .env vars are consumed by the Android client / tests, not the API. The Go API's /v1/rutracker/login endpoint takes the username/password **from the client request body**, logs in to rutracker on the client's behalf, and hands the bb_session cookie back for the client to replay on subsequent search calls.

Net contract: search needs BOTH a valid Lava-Auth key AND a rutracker session cookie. Missing either → **401**. Present-but-invalid cookie → **200 with empty results**.

2. Reproduction (captured HTTP status / body)

All against https://127.0.0.1:8443. Lava-Auth value derived from .env LAVA_AUTH_ACTIVE_CLIENTS (client android-1.2.7-1027, 16-byte UUID) — value never printed.

#	Request	Result
T1	GET /providers (no auth)	200 ; rutracker authType=CAPTCHA_LOGIN, caps SEA
T2	GET /v1/rutracker/search?query=1080p — no Lava-Auth	401 {"error": "unauthorized"}
—	host → https://rutracker.org/forum/index.php (IPv4)	200 , 1.07s, ip 172.67.182.196
—	host → /forum/tracker.php?nm=1080p anon (no cookie)	302 , 0 bytes
T3	search + valid Lava-Auth, no Auth-Token	401 {}

#	Request	Result
T4	search + valid Lava-Auth + bogus Auth-Token: rutracker:cookie:bb_session=... fake	200 {"provider":"rutracker","page":1,"totalPa ts":[]}
T5	search + valid Lava-Auth + legacy bare bogus cookie	200 , empty results
T6a	POST /v1/rutracker/login + valid Lava-Auth + .env creds	200 {"success":true,"authToken":"bb_session=... captcha needed)
T6b	search + valid Lava-Auth + REAL bb_session (from a live login)	200; 50 results; page 1/10; first title Другая кро decoded)

T6b is the load-bearing result: with both headers valid, the Go API returns 50 real rutracker torrents. Candidate **(c) "Go API can't scrape rutracker"** is **definitively refuted**.

3. Confirmed root cause

The Go API rutracker search endpoint is server-side-correct. It fails ("Search failed" = HTTP **401**) only because the request reaching it is missing one of its two required auth inputs. This is an **auth / header-forwarding** problem on the request INTO the Go API — NOT a server-side rutracker scrape failure (T6b = 50 results) and NOT a network/endpoint reachability problem (host reaches rutracker; /providers reached the API in T1).

The two 401 producers, with **distinct, diagnostic response bodies**:

- **401 {"error":"unauthorized"}** ⇒ the Lava-Auth per-instance key is missing/invalid on the request (**candidate b**). This is exactly the cold-start gap documented in core/domain/.../RepopulateProvidersOnStartupUseCase.kt:140-149 ("dynamic ApiBackedTrackerClients built at cold start have no Lava-Auth value → /v1/{provider}/search returns 401").
- **401 {}** ⇒ the request carried a valid Lava-Auth but **no rutracker session cookie** in Auth-Token (the ErrUnauthorized short-circuit).

This happens if the on-device login stored a session but the goapi search flow does not attach it as Auth-Token: rutracker:cookie:....

UNCONFIRMED: which of the two the keystone device actually hits — that needs the on-device HTTP capture (Chucker) of the failing /v1/rutracker/search response **body**. The two bodies above disambiguate it in one look. Given the client-side heal code that already exists specifically for the Lava-Auth key (RepopulateProvidersOnStartupUseCase.restoreActiveEndpointKey / reconcileActiveEndpoint), candidate **(b)** is the leading suspect; the cookie-forwarding branch is the secondary must-verify.

4. Proposed fix

4.0 One-command disambiguation (do this first)

On the failing keystone run, capture the /v1/rutracker/search **response body**:

- body {"error": "unauthorized"} → go to §4.1 (Lava-Auth key).
- body {} → go to §4.2 (rutracker cookie).
- body {"provider": "rutracker", ..., "results": []} (a 200) → session invalid/expired; re-login on device (the cookie is stale), not an auth-wiring bug.

4.1 If the key is missing (candidate b) — client-side

The Go API is correct; the fix is ensuring the search request carries the key. Verify the cold-start restore path in core/domain/src/main/kotlin/lava/domain/usecase/RepopulateProvidersOnStartupUseCase.kt actually populates Endpoint.GoApi.key onto the endpoint that the search flow resolves via NetworkApiRepositoryImpl.endpoint() AND onto the dynamic ApiBackedTrackerClient (activator.activate(apiBaseUrl, goApi.key), line 103). restoreActiveEndpointKey() (line 164) is a no-op when apiKeyProvider.keyFor(host, port) returns null (api-app not co-installed / permission denied) — confirm on the keystone device that the api-app key ContentProvider actually returns the key.

4.2 If the cookie is not forwarded — client-side

core/network/impl/.../NetworkApiRepositoryImpl.kt:124-135
getAuthHeader(token) returns "Auth-Token" to token for
Endpoint.GoApi. Confirm the goapi search flow (a) has a non-empty
stored rutracker session token, and (b) formats it as the Go API expects
— ParseAuthToken accepts either a legacy bare cookie (bb_session=...) OR
the typed rutracker:cookie:bb_session=... (multiprovider.go:41-50).
Note the Go login response (/v1/rutracker/login) returns the **full Set-
Cookie line** (bb_session=...; expires=...; Max-Age=...; path=...; domain=...;
secure; HttpOnly) as authToken. UNCONFIRMED: whether the client stores/
forwards that entire line vs. just bb_session=<value>; forwarding the
whole line puts cookie *attributes* into the request Cookie header. It
worked in T4/T5 with a clean bb_session=...; recommend the client store
only bb_session=<value> to avoid any ambiguity.

4.3 Optional Go-side hardening (not the root cause)

internal/handlers/v1/handlers.go:184-185 maps
provider.ErrUnauthorized to 401 {}. Emitting a discriminating body
(e.g. {"error":"login_required"}) would let the client (and this QA
harness) tell the "no rutracker session" 401 apart from the "no Lava-
Auth key" 401 without reading source. File+change: writeProviderError
ErrUnauthorized branch → writeJSON(c, 401,
gin.H{"error":"login_required"}). This is a diagnostics improvement,
not a functional fix.

5. Container state / cleanup

Left the Go API UP for further keystone work: lava-api-go (:8443,
healthy) + lava-postgres (healthy). Run ./stop.sh to tear down. No
other projects' containers were touched. No source was edited
(investigation only). Scratch under /run/media/milosvasic/
DATA4TB/.build-tmp/goapi-search-rca/ (token-bearing files deleted).
.env credentials were used only at runtime and never written to any
file; the live bb_session obtained during T6 will expire naturally.